



**IES
CO
MER
CIO**

Proyecto Fin de Grado

FP GS Desarrollo de Aplicaciones Multiplataforma

2025 - 2026



QuickTix

Autor:

Santiago Puebla Mendoza

Tipo de proyecto:

Gestión y venta de entradas.

Supervisor del TFG:

M^a Luz Ancín Rodríguez

Sumario

Índice.....	3
1. Introducción	6
1.1 Contexto y motivación	6
1.1.1 Problemas detectados en el flujo tradicional	6
1.1.2 Qué pretende resolver el proyecto	7
2. Objetivos	8
2.1 Objetivo general	8
2.2 Objetivos funcionales	8
2.3 Objetivos no funcionales	10
2.4 Objetivos de validación (pruebas)	10
3. Fases del proyecto y metodología	10
3.1 Metodología de desarrollo	10
3.2 Gestión de la carga de trabajo	11
3.3 Decisiones técnicas	12
3.3.1 WPF como cliente de escritorio	12
3.3.2 .NET MAUI como cliente móvil	12
3.3.3 ASP.NET Core Web API como backend	13
3.3.4 SQL Server + EF Core (Code First)	13
4. Alcance	14
4.1 Incluido	14
4.2 Fuera de alcance o pendiente de confirmar	14
5. Diseño de la solución	14
5.1 Arquitectura general	14

5.2 Estructura de proyectos	14
5.3 Responsabilidad por capa.....	15
5.3.1 Core.....	15
5.3.2 Contracts	16
5.3.3 API	18
5.3.4 DAL	19
Qué engloba	20
5.3.5 Frontend.Desktop	21
1) QuickTix.Desktop.LoginView	22
2) QuickTix.Desktop.UsersView	23
3) QuickTix.DesktopClientsView.....	24
4) QuickTix.Desktop.PricingView.....	26
5) QuickTix.Desktop.SalesView	27
5.3.6 Frontend.Mobile	28
1) Views/LoginPage.xaml	29
2) Views/ChangePasswordPage.xaml	30
3) Views/TicketsPage.xaml	30
4) Views/SubscriptionsPage.xaml	31
6. Bases de datos y modelo de dominio	32
6.1 Entidades principales	32
6.2 Integridad y reglas de negocio	32
6.3 Migraciones y semillas	32
7. API REST	33
7.1 Rutas Endpoint (ApiRoutes)	33
8. Implementación, configuración y despliegue	35
8.1 Compilación y ejecución local	35

8.1.1 Despliegue de SQL Server en Docker Desktop	35
8.1.2 Aplicación de migraciones EF Core	35
Instalación de la herramienta (si no está disponible)	35
Ejecutar migraciones	36
Connection string esperada (orientativa)	36
8.1.3 Arranque de la API y seed inicial	36
8.1.4 Ejecución de clientes	37
9. Conclusiones y trabajo futuro	37
9.1 Trabajo futuro y líneas de mejora	38
1) Consolidación técnica (calidad y robustez)	38
2) Mejora estética y experiencia de usuario (prioridad en Mobile)	38
3) Nuevas funcionalidades centradas en el cliente	38

1. Introducción

QuickTix es una solución software orientada a la gestión operativa y administrativa de la venta de entradas y abonos para recintos.

El sistema se compone de una API central y dos clientes: un cliente de escritorio (WPF) para perfiles administrativos y un cliente móvil (.NET MAUI) para el flujo operativo y de usuario final.

1.1 Contexto y motivación

Tras tres veranos consecutivos trabajando en la venta de entradas para las piscinas municipales de una localidad, pude comprobar de primera mano las limitaciones de un modelo de gestión tradicional basado en papel, anotaciones manuales y procedimientos repetitivos. En ese contexto, la prioridad no era únicamente “vender entradas”, sino mantener el ritmo operativo en momentos de alta afluencia, reducir errores humanos y disponer de un registro fiable de lo que se vendía, a quién y en qué condiciones.

La idea de QuickTix surge precisamente de esa experiencia: trasladar a una aplicación móvil (y a una interfaz de gestión) los procesos cotidianos de un recinto público, de forma que el personal pueda operar con rapidez, consistencia y trazabilidad, y que la administración disponga de información estructurada para control y análisis.

1.1.1 Problemas detectados en el flujo tradicional

1) Entradas en papel: coste, fragilidad y control deficiente

- ❑ **Coste y logística de impresión:** especialmente cuando se usan diseños con color, numeración o formatos específicos. Cada reposición implica tiempo, coordinación y gasto recurrente.
- ❑ **Fragilidad y deterioro:** el papel se rompe, se moja o se degrada fácilmente en un entorno como una piscina (agua, crema solar, calor), lo que dificulta tanto la entrega como el control.
- ❑ **Dificultad de recuento y conciliación:** el cierre de caja o el recuento diario se apoya en montones de tickets y anotaciones; es fácil que falten entradas, se dupliquen conteos o no coincidan importes con unidades vendidas.
- ❑ **Registro poco fiable:** cuando el registro depende de apuntes manuales, cualquier prisa o interrupción deriva en datos incompletos (faltan cantidades, importes, tipo de entrada, etc.).

2) Abonos y abonados: pérdidas, búsquedas lentas y falta de trazabilidad

- ❑ **Pérdida o deterioro del bono:** el usuario extravía el abono físico o llega en mal estado, generando discusiones, retrasos y necesidad de “verificar a ojo”.
- ❑ **Búsqueda manual de abonados:** localizar a una persona en listados impresos o anotaciones se vuelve lento, especialmente en horas punta o cuando hay usuarios con nombres similares.

- ❑ **Información dispersa:** en el modelo tradicional es habitual que el estado del abono (vigente, consumido, caducado, etc.) quede implícito o se marque a mano, con margen elevado de error.

3) Procedimientos manuales repetitivos: cuello de botella en horas punta

- ❑ **Sellar o escribir la fecha a mano** en cada entrada (y posteriormente romper/arrancar y entregar al cliente) es un proceso lento por naturaleza.
- ❑ En escenarios de alta demanda (por ejemplo, **campamentos o grupos grandes**, 50–80 personas), ese proceso se convierte en un auténtico cuello de botella: aumenta el tiempo de espera, la presión sobre el personal y la probabilidad de errores.
- ❑ **Gestión de excepciones** (entradas especiales, descuentos, invitaciones, incidencias con abonos) se resuelve “sobre la marcha”, sin un criterio consistente ni un registro claro.

4) Falta de centralización: datos no estructurados y difícil toma de decisiones

- ❑ Sin un sistema central, los datos quedan repartidos entre papel, anotaciones y memoria operativa del personal.
- ❑ Esto impide responder con facilidad a preguntas básicas de gestión:
 - ❑ cuántas entradas se han vendido por tipo y día,
 - ❑ qué importe se ha recaudado,
 - ❑ qué porcentaje corresponde a abonos,
 - ❑ qué reglas de precio se aplicaron y por qué,
 - ❑ quién realizó la venta y en qué horario.

1.1.2 Qué pretende resolver el proyecto

QuickTix aborda la necesidad de **centralizar** los procesos habituales en instalaciones públicas: configuración de recintos (*venues*), definición de tipos de entrada y abonos, aplicación de precios y descuentos por contexto, y registro de ventas e históricos. La solución pone el foco en:

- ❑ **Operativa rápida** desde un dispositivo móvil (venta y validación ágil).
- ❑ **Trazabilidad y control** (histórico de ventas, consistencia en precios, responsabilidad por usuario/rol).
- ❑ **Mantenibilidad y evolución** mediante una arquitectura modular que permita ampliar reglas de precios, incorporar nuevas modalidades de entrada/abono y adaptarse a distintos recintos sin rehacer el sistema.

2. Objetivos

QuickTix nace de la necesidad de disponer de una plataforma única para la **gestión operativa y administrativa de recintos** y la **venta controlada de entradas y abonos**, minimizando fricciones en la operativa diaria (taquilla/gestión) y garantizando trazabilidad, seguridad por roles y coherencia en la aplicación de precios.

En entornos reales, los recintos suelen enfrentarse a casuísticas recurrentes: múltiples tipos de acceso (entrada suelta, abono), precios condicionados por contexto (p. ej., temporada, tipo de día, reglas internas), gestión de usuarios con distintos permisos y la necesidad de consultar históricos para auditoría o cierre de caja. A esto se suma la conveniencia de ofrecer interfaces diferenciadas según el perfil: una interfaz de gestión (administración) y una interfaz de uso operativo (venta/consulta), manteniendo una capa centralizada que concentre la lógica de negocio y el acceso a datos.

El objetivo del proyecto es, por tanto, desarrollar una solución que combine:

- ❑ una **API** como núcleo de negocio y datos,
- ❑ y **clientes** (escritorio y/o móvil) para cubrir los escenarios de gestión y operación, con un diseño modular y extensible que permita añadir reglas de precios, nuevos flujos y mejoras sin comprometer la mantenibilidad.

2.1 Objetivo general

El objetivo general del proyecto es desarrollar una plataforma de gestión y venta (QuickTix) que permita **administrar recintos, usuarios y catálogo, y realizar ventas de entradas y abonos**, garantizando:

- ❑ una experiencia de uso clara para perfiles no técnicos,
- ❑ la aplicación consistente de reglas de precios por contexto,
- ❑ la trazabilidad completa de la actividad (ventas e histórico),
- ❑ y un modelo de seguridad basado en roles que limite acciones según el perfil.

Esta solución no se limita únicamente a “registrar ventas”, sino que pretende servir como herramienta de trabajo para el día a día: configuración previa (recintos, catálogo, pricing) y operación (venta y consulta), de forma que la plataforma sea escalable a nuevos recintos, tipos de acceso o reglas comerciales sin necesidad de reestructuraciones profundas.

2.2 Objetivos funcionales

Para alcanzar el objetivo general, el proyecto se articula en torno a los siguientes objetivos funcionales:

- ❑ **Gestión de recintos (Venue) y sus clientes (Client)**
 - ❑ Permitir el alta, edición y consulta de recintos.
 - ❑ Asociar clientes/usuarios a un recinto cuando proceda, facilitando la administración segmentada por centro.

- ❑ Asegurar que los gestores trabajan sobre los recintos asignados y que el acceso a datos sea coherente con ese ámbito.

❑ **Gestión del catálogo de entradas (Ticket) y abonos (Subscription)**

- ❑ Mantener un catálogo centralizado y consistente de tipos de entrada y modalidades de abono.
- ❑ Permitir que el catálogo sea reutilizable entre recintos, evitando duplicidad y favoreciendo estandarización, pero habilitando particularidades cuando sea necesario.

❑ **Registro de ventas (Sale) y consulta de histórico**

- ❑ Registrar cada venta con el nivel de detalle necesario para auditoría: tipo de producto (entrada/abono), cantidades, importes y metadatos relevantes.
- ❑ Proporcionar mecanismos para consultar el histórico con criterios habituales (por recinto, por fechas, por usuario/gestor, etc.), orientado a revisión y control.

❑ **Configuración de precios por recinto y aplicación de reglas por contexto**

- ❑ Permitir definir precios específicos por recinto tanto para entradas como para abonos.
- ❑ Incorporar “contextos” o reglas que modifiquen la aplicación del precio (por ejemplo, tipologías de acceso, categorías, condiciones del día, etc.), de manera que el sistema calcule el importe final de forma consistente y verificable.
- ❑ Preparar el sistema para evolución: nuevas reglas o tipos de tarificación deben poder integrarse sin romper el modelo.

❑ **Gestión de usuarios y roles (administrador y gestor)**

- ❑ Definir perfiles con permisos diferenciados:
 - ❑ **Administrador:** gestión global (catálogo, recintos, usuarios, configuración avanzada).
 - ❑ **Gestor:** operativa de su ámbito (venta y consulta de histórico de su recinto, mantenimiento operativo si aplica).
- ❑ Garantizar que las operaciones críticas (configuración, cambios estructurales) quedan restringidas a los roles adecuados.

2.3 Objetivos no funcionales

Aunque tu listado está centrado en lo funcional (correcto), conviene añadir objetivos no funcionales porque justifican decisiones de arquitectura:

- ❑ **Seguridad y control de acceso:** autenticación y autorización por roles; protección de endpoints y validación de operaciones según ámbito.
- ❑ **Mantenibilidad y extensibilidad:** separación de responsabilidades (API/Core/DAL y clientes), contratos claros (DTOs), y patrones que faciliten añadir módulos (p. ej., nuevas reglas de pricing).
- ❑ **Trazabilidad y robustez:** respuestas homogéneas y manejo centralizado de errores para que los clientes puedan mostrar mensajes coherentes y registrar incidencias.
- ❑ **Rendimiento en operativa:** consultas y listados eficientes (históricos, catálogo, pricing) para evitar fricción en escenarios de venta.

2.4 Objetivos de validación (pruebas)

- ❑ Validar los flujos principales en escenarios reales o simulados:
 - ❑ alta/configuración (recinto + catálogo + pricing),
 - ❑ venta (entrada y abono, individual y batch si aplica),
 - ❑ consulta de histórico,
 - ❑ control de permisos (admin vs gestor).
- ❑ Complementar con pruebas técnicas:
 - ❑ pruebas de integración de API (endpoints clave),
 - ❑ pruebas unitarias de reglas de pricing y validaciones de dominio.

3. Fases del proyecto y metodología

3.1 Metodología de desarrollo

Para un proyecto como QuickTix, con objetivos incrementales y con necesidad de validar el funcionamiento real “de punta a punta” (API + persistencia + cliente), resulta adecuado un enfoque iterativo inspirado en Scrum, pero aplicado de forma ligera. En lugar de intentar cerrar desde el inicio un alcance completo y rígido, se prioriza construir el sistema por incrementos funcionales que aporten valor tangible y permitan detectar temprano problemas de diseño, experiencia de usuario o integración.

En la práctica, el desarrollo se ha organizado alrededor de estos principios:

- ❑ **Backlog evolutivo:** las funcionalidades se descomponen en tareas y mejoras priorizadas. El backlog se alimenta a medida que surgen nuevas necesidades del

dominio (por ejemplo, casuísticas de venta, gestión de abonos, o ajustes de pricing por recinto) y conforme se validan flujos en escenarios reales.

- ❑ **Iteraciones cortas con entregables verificables:** cada iteración intenta cerrar un flujo completo (por ejemplo: “configurar recintos”, “gestionar catálogo”, “registrar una venta y consultarla”). Esto evita desarrollar capas aisladas sin comprobar su integración real.
- ❑ **Vertical slices (API + UI + datos):** el patrón de trabajo más habitual consiste en implementar, en la misma iteración:
 - ❑ endpoint en la API (contrato y validaciones),
 - ❑ persistencia en EF Core,
 - ❑ pantalla/flujo en el cliente (WPF o MAUI),
 - ❑ y pruebas manuales mínimas (Swagger/Postman/cliente) para validar el circuito completo.
- ❑ **Refactorización y consolidación:** conforme la solución crece, se introducen refactorizaciones periódicas para mantener consistencia:
 - ❑ consolidación de contratos y DTOs,
 - ❑ unificación de convenciones de endpoints,
 - ❑ fortalecimiento del manejo de errores y respuestas estándar,
 - ❑ y limpieza de componentes UI para reducir duplicidad.

Este enfoque es especialmente útil cuando el dominio real (venta en recinto público) presenta múltiples excepciones y escenarios no evidentes al inicio. La validación temprana de flujos ayuda a detectar rápidamente qué partes requieren más robustez (por ejemplo, pricing contextual, históricos, búsquedas de abonados, etc.).

3.2 Gestión de la carga de trabajo

La planificación y la organización del trabajo se han apoyado principalmente en el repositorio público de QuickTix, usando el apartado “**Projects**” de GitHub como soporte para anotar tareas, agrupar funcionalidades y reflejar el avance.

La intención original era documentar con más rigor:

- ❑ **Iteraciones/sprints:** fechas de inicio y fin, objetivo principal y entregables.
- ❑ **Estimación y horas dedicadas:** distribución por bloques (análisis, diseño, implementación, pruebas, documentación).
- ❑ **Riesgos/incidencias y mitigación:** por ejemplo, cambios de alcance, problemas de integración entre clientes, ajustes de pricing, decisiones de arquitectura, etc.

Sin embargo, al tratarse de un proyecto desarrollado en paralelo al aprendizaje y a la exploración técnica, el registro no ha mantenido una trazabilidad constante y completa. Parte del seguimiento se ha diluido en commits, ramas, notas y tareas que se resolvían directamente durante la implementación.

Aun así, el enfoque iterativo permitió mantener control efectivo del progreso mediante:

- ❑ **Commits frecuentes** que reflejan avances incrementales.
- ❑ **Bloques funcionales claramente identificables** (por ejemplo, módulos de venta/históricos, catálogo, usuarios/roles, pricing, etc.).
- ❑ **Validación continua** en clientes y API, ajustando rápidamente lo que no funcionaba o lo que se podía simplificar.

De cara a una mejora futura del proceso, se propone:

- ❑ mantener milestones por grandes bloques (MVP, ventas, pricing, gestión avanzada),
- ❑ y registrar al menos un resumen por iteración (objetivo, cambios principales, incidencias).

3.3 Decisiones técnicas

Las decisiones tecnológicas de QuickTix están orientadas a un objetivo principal: **construir una solución operativa y mantenible**, con un stack homogéneo, productivo y coherente para backend y clientes. A continuación se detallan las decisiones más relevantes y su justificación:

3.3.1 WPF como cliente de escritorio

Se ha elegido WPF como cliente de escritorio por su madurez y su adecuación a interfaces de gestión:

- ❑ Permite construir de forma eficiente **paneles administrativos y pantallas CRUD** (recintos, usuarios, catálogo, precios).
- ❑ Ecosistema consolidado en Windows, con herramientas estables en Visual Studio.
- ❑ Encaja de forma natural con el patrón **MVVM**, facilitando separación de vista y lógica de presentación, y mejorando mantenibilidad.

3.3.2 .NET MAUI como cliente móvil

Para el cliente móvil se ha elegido .NET MAUI por:

- ❑ **Reutilización de stack .NET** y posibilidad de compartir patrones, contratos y parte de la lógica entre aplicaciones.

- ❑ Compatibilidad con MVVM y con el enfoque de binding, lo que ayuda a mantener una estructura similar a la de WPF.
- ❑ Integración directa en Visual Studio: simplifica desarrollo paralelo de escritorio y móvil en un mismo entorno, reduciendo fricción de tooling.
- ❑ Adecuación a escenarios de **operativa en movilidad**, especialmente si el objetivo es vender o gestionar accesos desde el teléfono en el propio recinto.

3.3.3 ASP.NET Core Web API como backend

Se utiliza ASP.NET Core Web API como núcleo de la plataforma por:

- ❑ Robustez y rendimiento en producción.
- ❑ **Inyección de dependencias nativa**, facilitando arquitectura modular, testeable y extensible.
- ❑ Ecosistema amplio de middlewares (autenticación/autorización, logging, manejo de excepciones, CORS, etc.).
- ❑ Facilidad para exponer contratos REST consumibles por múltiples clientes (WPF/MAUI y posibles extensiones futuras).

3.3.4 SQL Server + EF Core (Code First)

Se ha optado por SQL Server con EF Core en enfoque Code First por:

- ❑ Integración natural con el ecosistema .NET y productividad en el desarrollo.
- ❑ **Migraciones** como mecanismo controlado para evolucionar el esquema de datos en paralelo al código, manteniendo coherencia entre versiones.
- ❑ Adecuación para datos transaccionales y relacionales como:
 - ❑ ventas e histórico,
 - ❑ usuarios/roles,
 - ❑ catálogo de productos (entradas/abonos),
 - ❑ configuración de precios y reglas por recinto.

Además, EF Core facilita:

- ❑ un modelo de entidades coherente con el dominio,
- ❑ validaciones y relaciones,
- ❑ y una iteración rápida al incorporar nuevos requisitos.

4. Alcance

4.1 Incluido

- ❑ API REST para recursos principales (User, Admin, Manager, Client, Venue, Ticket, Subscription, Pricing, Sale).
- ❑ Persistencia en base de datos relacional mediante EF Core.
- ❑ Cliente de escritorio para administración/gestión con pantallas tipo CRUD.
- ❑ Cliente móvil para flujos operativos (venta/consulta de tickets y abonos).

4.2 Fuera de alcance o pendiente de confirmar

- ❑ Integración con pasarela de pago real (si aplica).
- ❑ Operación offline en móvil.
- ❑ Observabilidad avanzada (métricas, trazas distribuidas) más allá de logging básico.
- ❑ Cobertura de tests alta y pipeline CI/CD completo (a completar).

5. Diseño de la solución

5.1 Arquitectura general

QuickTix adopta una arquitectura en capas con separación de responsabilidades: la API expone endpoints y orquesta servicios; Core define dominio y reglas; DAL implementa persistencia; y los clientes consumen la API mediante HTTP.

5.2 Estructura de proyectos

Proyecto	Responsabilidad	Notas
QuickTix.API	Web API. Controladores, filtros y configuración.	Expone endpoints REST y aplica filtros globales.
QuickTix.Core	Dominio e interfaces.	Entidades, reglas; servicios de negocio que tengan que ver con el dominio.
QuickTix.DAL	Acceso a datos (EF Core).	DbContext, migraciones y repositorios.
QuickTix.Contracts	Contratos compartidos.	DTOs, rutas centralizadas, enums y respuesta genérica ApiResponse.
QuickTix.Desktop	Cliente WPF (MVVM).	Gestión administrativa/operativa con pantallas CRUD.

QuickTix.Mobile

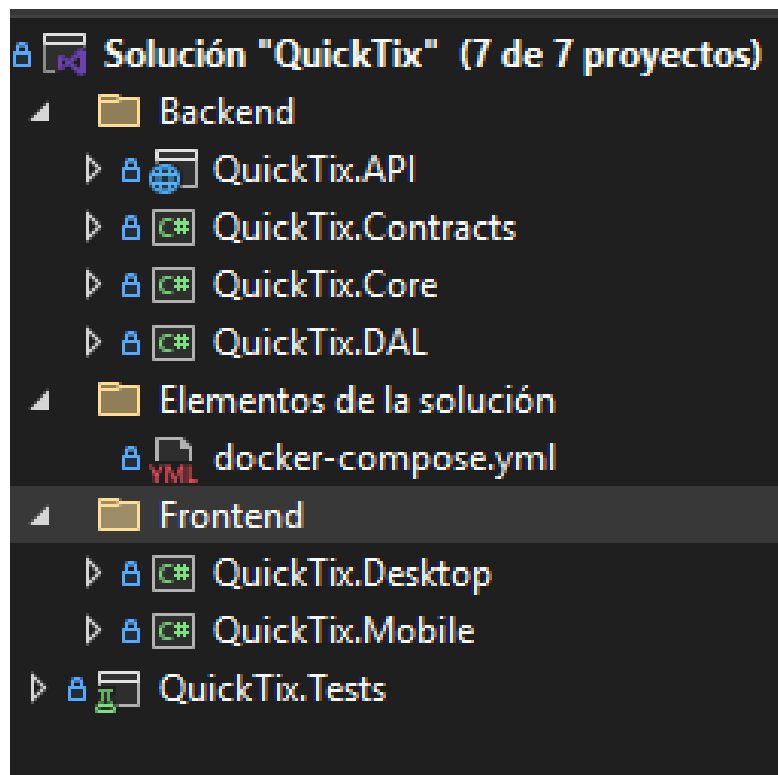
Cliente MAUI (MVVM).

Flujos móviles orientados a usuario final y manager.

QuickTix.Tests

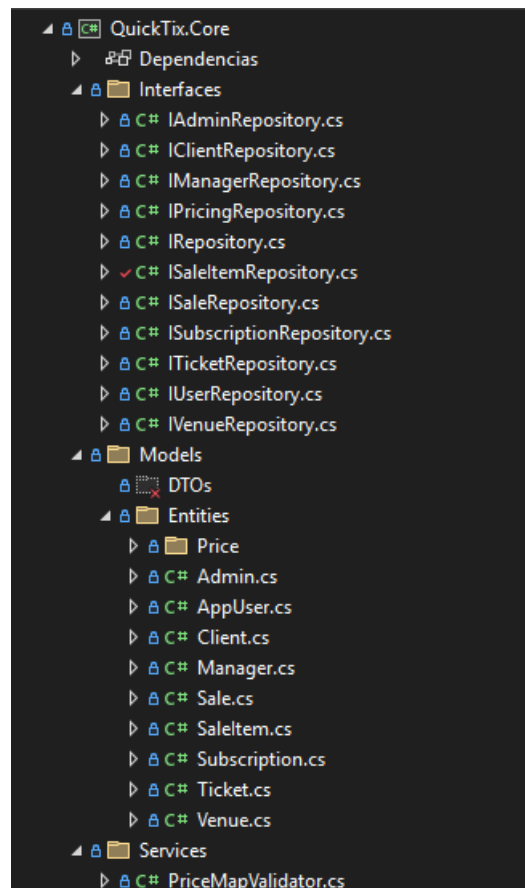
Pruebas xUnit.

Base para unit e integration tests.



5.3 Responsabilidad por capa.

5.3.1 Core



La capa **Core** representa el **núcleo funcional y de dominio** de QuickTix. Su responsabilidad es definir “qué es el sistema” y “qué reglas debe cumplir”, manteniéndose **independiente** de detalles de infraestructura (HTTP, UI, base de datos concreta). En términos prácticos, Core actúa como el contrato estable entre la API/clientes y la persistencia.

1. Modelo de dominio

- Define las entidades principales y sus relaciones (por ejemplo: usuarios/roles de negocio, recintos, tickets, abonos, ventas y líneas de venta).

- ❓ Establece invariantes del dominio: qué significa una venta válida, cómo se compone un catálogo, qué datos son obligatorios, etc.

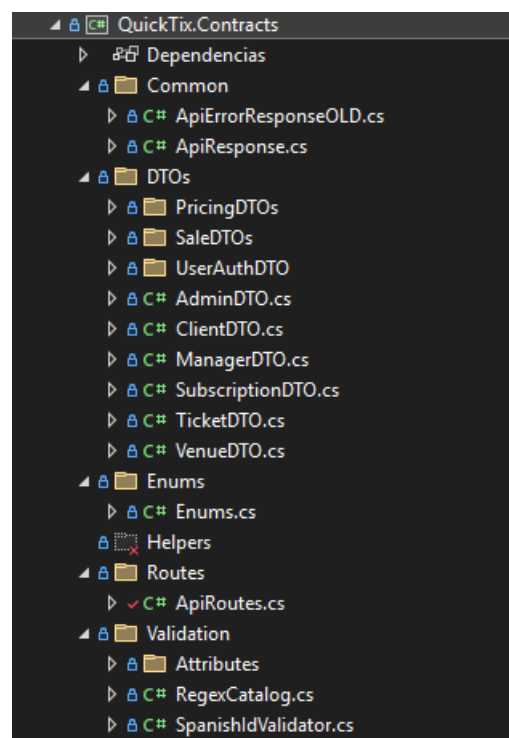
2. Contratos de acceso a datos y operaciones

- ❓ Declara interfaces (repositorios/servicios) que expresan **qué necesita** el dominio para funcionar (consultar/guardar ventas, obtener pricing, administrar recintos...), sin decidir **cómo** se implementa.
- ❓ Esto permite que la API dependa de abstracciones y que la implementación viva en DAL, favoreciendo testabilidad y desacoplamiento.

4. Reglas y validadores de negocio

- ❓ Contiene lógica que no es propia de la UI ni de la API: validaciones de pricing, coherencia de mapas de precios, restricciones de venta, consistencia de catálogos, etc.

5.3.2 Contracts



La capa **Contracts** define el **contrato público de comunicación** entre la API y sus consumidores (QuickTix.Desktop, QuickTix.Mobile y cualquier cliente futuro). Su objetivo es fijar, de forma estable y versionable, **qué datos viajan por la red y en qué forma**, evitando que los clientes dependan de entidades internas del dominio o de detalles de persistencia.

En términos de arquitectura, esta capa actúa como el “lenguaje común” del sistema:

- ❓ **DTOs:** modelos de entrada/salida para endpoints (por recurso y por caso de uso), evitando exponer directamente las entidades del Core.
- ❓ **Routes:** centralización de rutas/paths para que los clientes consuman endpoints de forma consistente y con menos errores por hardcoded.
- ❓ **Common:** modelos transversales compartidos (respuestas estándar, errores, utilidades comunes).
- ❓ **Validation/Attributes/Helpers:** validaciones y utilidades que se pueden reutilizar tanto en API como en clientes, manteniendo coherencia de reglas de formato (por ejemplo, validaciones de campos y catálogos).

La idea clave es que **Contracts es “estable”**: si el Core o la DAL evolucionan, los clientes no deberían romperse mientras el contrato se mantenga o se versionen cambios.

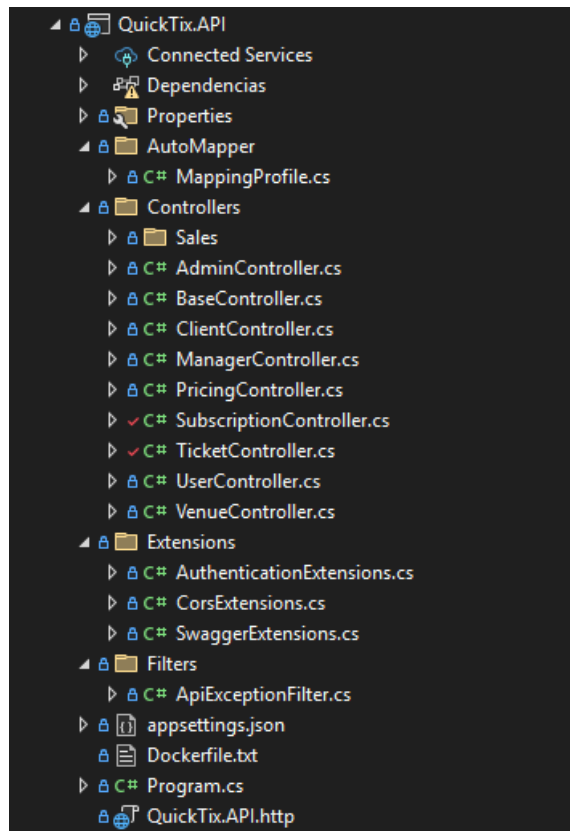
En esta clase vive `ApiResponse<T>`, que estandariza todas las respuestas de la API en un contenedor común, incluyendo `StatusCode`, un flag `IsSuccess`, un listado de `ErrorMessages`, el `Result` tipado y un `TraceId` para trazabilidad. Esto permite que Desktop/Mobile manejen éxitos y errores de forma uniforme, simplificando el tratamiento de fallos y facilitando soporte (el usuario/reporting puede aportar el `TraceId` para localizar el error en logs).

```
namespace QuickTix.Contracts.Common
{
    44 referencias
    public class ApiResponse<T>
    {
        6 referencias
        public HttpStatusCode StatusCode { get; set; }
        7 referencias
        public bool IsSuccess { get; set; } = true;
        7 referencias
        public List<string> ErrorMessages { get; set; } = new();
        10 referencias
        public T? Result { get; set; }
        2 referencias
        public string? TraceId { get; set; }

        15 referencias
        public static ApiResponse<T> Ok(T result, HttpStatusCode statusCode = HttpStatusCode.OK, string? traceId = null)
            => new()
        {
            StatusCode = statusCode,
            IsSuccess = true,
            Result = result,
            TraceId = traceId
        };

        10 referencias
        public static ApiResponse<T> Fail(HttpStatusCode statusCode, IEnumerable<string> errors, string? traceId = null)
            => new()
        {
            StatusCode = statusCode,
            IsSuccess = false,
            Result = default,
            ErrorMessages = errors?.ToList() ?? new List<string> { "Error no especificado." },
            TraceId = traceId
        };
    }
}
```

5.3.3 API



1. Exposición de endpoints REST

- ❑ Definir rutas y verbos (GET/POST/PUT/PATCH/DELETE) para los recursos de QuickTix.
- ❑ Mantener consistencia en naming, códigos HTTP y estructura de respuesta.

2. Seguridad y control de acceso

- ❑ Aplicar autenticación y autorización (por roles) a nivel de controlador/acción.
- ❑ Evitar que un rol acceda a recursos fuera de su ámbito (por ejemplo, un manager manipulando recintos no asignados, si ese control existe en Core/servicio).

3. Validación “de borde”

- ❑ Validar que el request es consistente: DTOs obligatorios, formatos, rangos, parámetros de ruta, etc.

4. Orquestación

- ❑ La API llama a repositorios a través de interfaces (Core/DAL).

- ❓ Evita cálculos complejos o reglas de pricing dentro del controller; el controller coordina y devuelve resultados.

5. Contratos y mapeo

- ❓ No debe devolver entidades EF directamente.
- ❓ Mapea **Entity ↔ DTO** usando AutoMapper u otra estrategia para desacoplar el dominio de la capa HTTP.

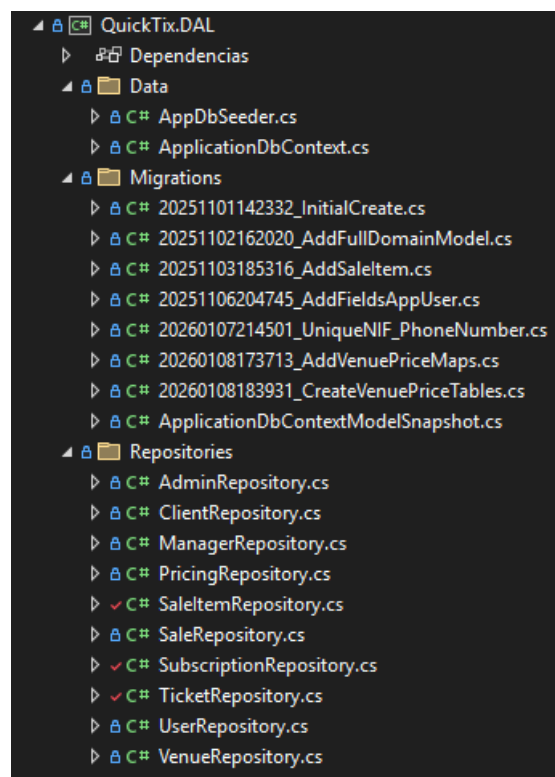
6. Respuestas y errores consistentes

- ❓ Devolver una estructura homogénea de éxito/error.
- ❓ Centralizar excepciones mediante filtros/middlewares para evitar “try/catch” repetidos y asegurar trazabilidad (traceld, mensaje, status code).

En esta clase también vive BaseController, que centraliza el uso de todos los Controllers por entidad para operaciones CRUD, aplicando el contrato de la clase ApiResponse<T> para todos.

(Ver DOC para más)

5.3.4 DAL



La capa **DAL (Data Access Layer)** es la responsable de la **persistencia y acceso a datos** de QuickTix. Su función es materializar, con una tecnología concreta, los contratos definidos en Core (interfaces de repositorio) y proporcionar a la aplicación una forma consistente, testable y mantenible de leer y escribir información en la base de datos.

En términos prácticos, DAL:

- ❑ **Implementa el acceso a base de datos** mediante **EF Core**, encapsulando consultas, seguimiento de cambios y operaciones CRUD.
- ❑ **Traduce operaciones de dominio a operaciones de persistencia**, resolviendo detalles que no deben existir en la API ni en Core (tracking, includes, transacciones, rendimiento de consultas, etc.).
- ❑ **Gestiona la evolución del esquema** con **migraciones** y mantiene un snapshot del modelo para trazabilidad y despliegue.
- ❑ **Centraliza inicialización/seed** de datos cuando procede (por ejemplo, usuarios iniciales, catálogos base o datos mínimos de arranque).

Dicho de otro modo: Core define *qué se necesita hacer* (interfaces, reglas y entidades) y la DAL define *cómo se hace* contra SQL Server usando EF Core.

Qué engloba

❑ Data

- ❑ **ApplicationDbContext**: definición del modelo EF, DbSet, configuración de entidades y relaciones.
- ❑ **AppDbSeeder**: inicialización/seed de datos para muestra inicial.

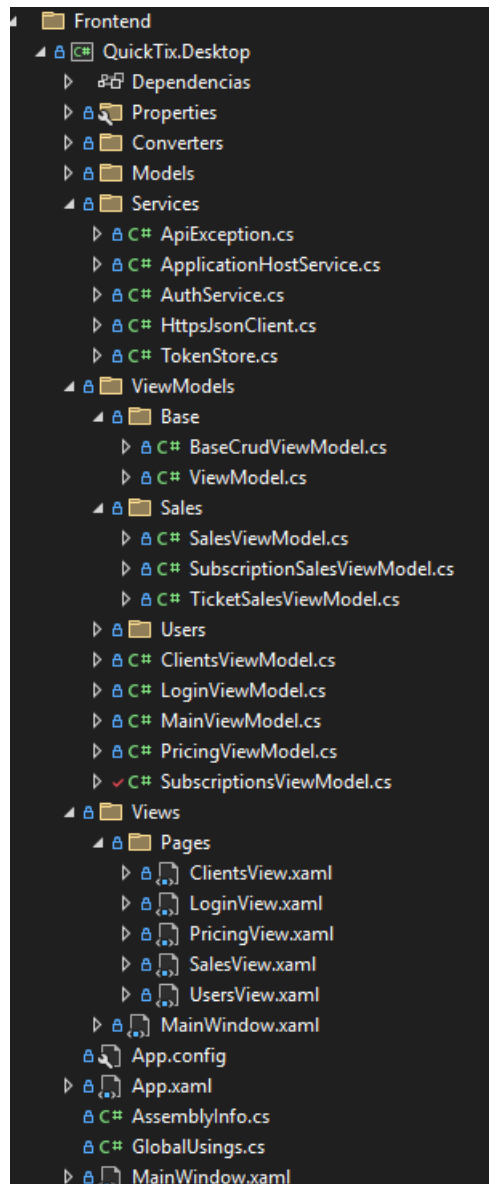
❑ Migrations

- ❑ **Historial de cambios de esquema** (creación inicial, ampliaciones del dominio, añadidos como SaleItem, constraints, pricing por venue, etc.).
- ❑ **ApplicationDbContextModelSnapshot**: estado actual del modelo para EF.

❑ Repositories

- ❑ **Implementaciones concretas** (VenueRepository, TicketRepository, SaleRepository, PricingRepository, etc.) que cumplen las interfaces de Core y encapsulan las queries/actualizaciones.

5.3.5 Frontend.Desktop



La capa **QuickTix.Desktop** es el **cliente de escritorio** orientado a la operación y administración interna del sistema. Su objetivo es proporcionar una interfaz eficiente para usuarios con roles de gestión (principalmente administradores y gestores), centralizando tareas de configuración y control que requieren pantallas con listados, formularios y visión global de datos.

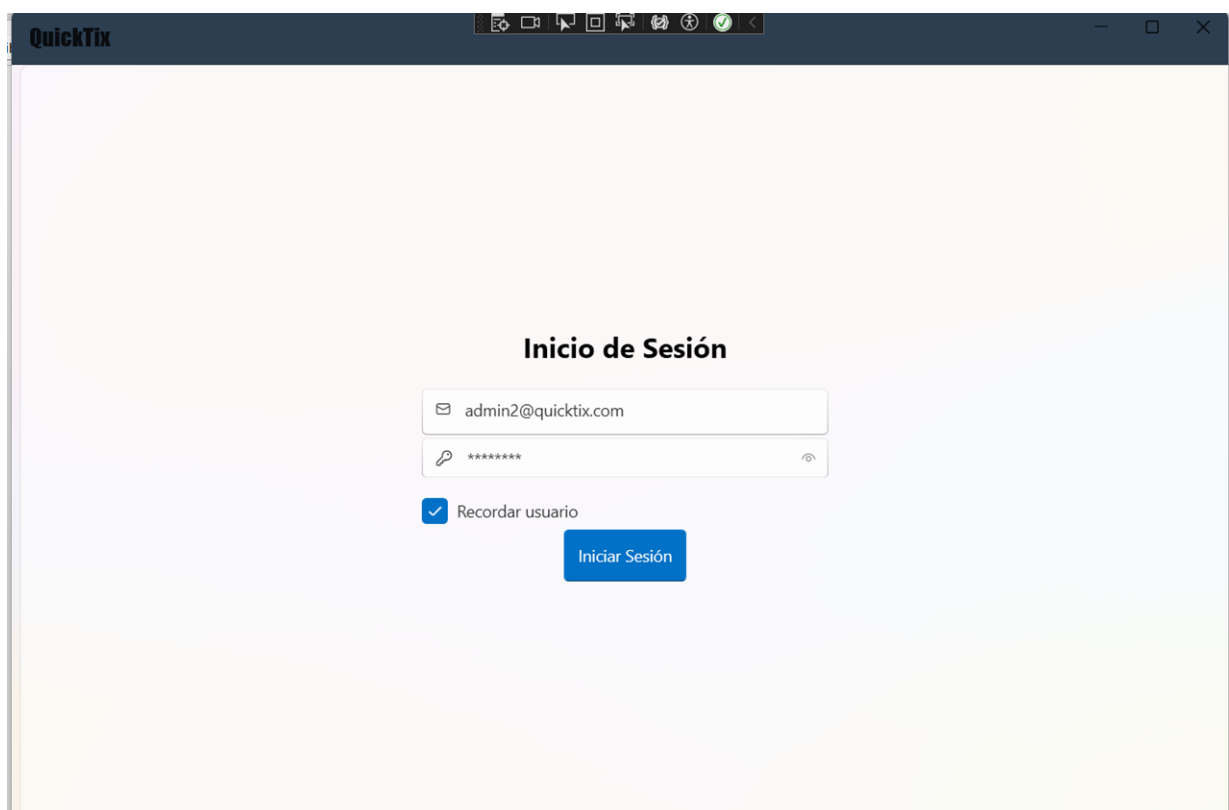
Funcionalmente, esta capa cubre:

- ❑ **Autenticación e inicio de sesión** contra la API y gestión de sesión (token/credenciales).
- ❑ **Paneles de administración (CRUD)** para entidades clave: usuarios internos, clientes y, cuando aplica, recintos/catálogos.

- ❓ **Configuración operativa avanzada**, como el **mapa de precios por recinto**, diferenciando claramente campos estructurales (no editables) y valores modificables.
- ❓ **Consulta y auditoría**, mediante pantallas de **histórico de ventas** y detalle de operaciones.

A nivel técnico, sigue un enfoque **MVVM**, desacoplando vistas (XAML) de la lógica de presentación (ViewModels), y consume la API mediante servicios HTTP centralizados, aplicando manejo uniforme de errores y validaciones para ofrecer una experiencia consistente.

1) QuickTix.Desktop.LoginView



Propósito: pantalla de acceso al sistema para usuarios internos (admin/gestor) desde el cliente WPF.

Qué hace:

- ❓ Recoge credenciales mediante dos campos:
 - ❓ **Username** (TextBox con icono de correo).
 - ❓ **Password** (PasswordBox con icono de llave).
- ❓ Permite activar **“Recordar usuario”** (RememberUser) para persistir el nombre de usuario y agilizar accesos posteriores.
- ❓ Ejecuta el inicio de sesión con el comando CheckLoginCommand.
Este comando valida credenciales contra la API y, si es correcto, almacena token/sesión y navega al contenedor principal (menú / main).

2) QuickTix.Desktop.UsersView

Administradores

ID	Nombre	Email	NIF	Teléfono
1	Administrador	admin2@quicktix.com	00000000A	625806446
2005	asdsad	fwe2f@df.com	45901396c	666666669

Gestores

ID	Nombre	Email	Recinto	NIF	Teléfono
1	Gestor Piscina	manager@quicktix.com	Piscina Municipal de Nalda		
1003	paco	paco@paco.com	Piscina Municipal de Nalda	123323123s	
2008	jopa	j@jj.com	Piscina Municipal de Nalda	12312312j	222222222
3007	w	0@0.com	Piscina Municipal de Nalda	12312311q	098709878

Propósito: administración de usuarios internos, separando claramente la gestión de **Administradores** y **Gestores**.

Qué hace:

- ❓ Divide la pantalla en dos secciones:
 1. **Administradores** (parte superior)

2. Gestores (parte inferior)

❓ Cada sección ofrece un mini-CRUD:

1. listado (ListView) con selección,
2. **Añadir, Editar y Eliminar** (habilitados solo si hay selección).

❓ Para alta/edición utiliza **popups (flyouts)** con formularios:

1. **Admin**: nombre, NIF, email, teléfono.
2. **Manager**: además de lo anterior, permite **asignar Recinto (Venue)** mediante un ComboBox (esto refleja la relación “gestor trabaja sobre un recinto”).

❓ Muestra mensajes de error por sección (AdminsVM.ErrorMessage, ManagersVM.ErrorMessage), lo que sugiere validación de formulario y feedback inmediato al usuario.

3) QuickTix.DesktopClientsView

The screenshot displays the 'QuickTix.DesktopClientsView' interface. It is divided into two main sections: 'Clientes' on the left and 'Detalle del cliente' on the right.

Clientes Section:

- Buttons: 'Añadir Cliente', 'Eliminar seleccionado', 'Editar seleccionado'.
- Table:

ID	Nombre	Teléfono
1	Cliente 1	3
2	Cliente 2	123123321
5	joselito	aaaaaaaaa
6	sgsdgsrrrr	023i10
1004	papu	
1005	epa	01010101010
1006	aasd	123123123
1007	joseee	000000000
2005	epas	000111000

Detalle del cliente Section:

- Client Name: **joselito**
- Email: a@a.com
- ID: 5
- NIF: aaaaaaaaa
- Telefono: aaaaaaaaa
- Email: a@a.com
- Buttons: 'Abonos', 'Entradas', 'Usuarios relacionados', 'Vender abono', 'Cancelar abono'.
- Table:

ID	Categoría	Duración	Inicio	Fin
13	Adulto	Mensual	1/2/2026 12:00:00 At	2/2/2026 12:00:00 At
16	Adulto	Mensual	1/2/2026 12:00:00 At	2/2/2026 12:00:00 At
7	Jubilado	Quincenal	12/27/2025 12:00:00	1/11/2026 12:00:00 F
8	Niño	Mensual	12/27/2025 12:00:00	1/27/2026 12:00:00 F

Propósito: gestión de clientes (abonados/usuarios finales del recinto) con un enfoque **master-detail** y operaciones asociadas (especialmente abonos).

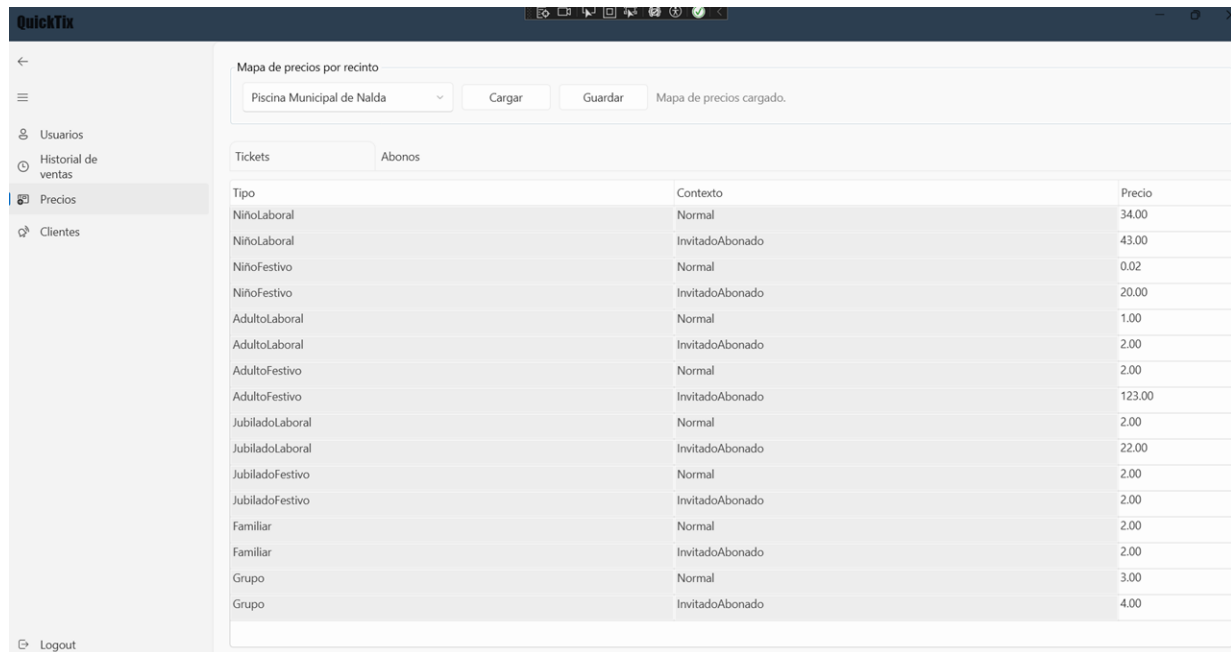
Qué hace:

❓ **Panel izquierdo (Clientes):**

- ❑ Listado de clientes con columnas: ID, Nombre, Teléfono.
- ❑ Acciones principales:
 - ❑ **Añadir Cliente** (abre popup).
 - ❑ **Editar seleccionado** (abre popup precargado).
 - ❑ **Eliminar seleccionado** (ejecuta borrado por Id).
- ❑ **Panel derecho (Detalle del cliente):**
 - ❑ Muestra cabecera con datos del cliente seleccionado (nombre, email) y un bloque de “ficha” (id, teléfono, NIF, email).
 - ❑ Si no hay selección, muestra un mensaje indicando que se seleccione un cliente.
- ❑ **Pestañas funcionales:**
 - ❑ **Abonos:**
 - ❑ Listado de abonos del cliente (SubscriptionsVM.Items) con datos completos: categoría, duración, fechas inicio/fin, precio y estado.
 - ❑ Acciones:
 - ❑ **Vender abono** (abre popup de venta/alta).
 - ❑ **Cancelar abono** (habilitado solo si hay un abono seleccionado).
 - ❑ **Entradas**
 - ❑ **Usuarios relacionados**

Esta vista, en conjunto, no solo hace CRUD del cliente, sino que actúa como **punto operativo** para “gestionar la relación cliente ↔ abonos” (venta, cancelación y consulta del histórico de abonos del cliente).

4) QuickTix.Desktop.PricingView



Propósito: configurar el **mapa de precios por recinto** (pricing por Venue) de forma centralizada y editable.

Qué hace:

- ❓ Barra superior para trabajar por recinto:
 - ❓ Selección de **Venue** en ComboBox.
 - ❓ Botón **Cargar** (trae el mapa de precios desde la API).
 - ❓ Botón **Guardar** (persiste cambios).
 - ❓ Mensaje informativo (InfoMessage) para feedback (estado de carga/guardado).
- ❓ Visualización de errores (ErrorMessage) cuando algo falla (API, validación, etc.).
- ❓ Edición de precios mediante pestañas y DataGrids:
 - ❓ **Tickets:**
 - ❓ Tabla de precios por combinación **Tipo + Contexto**.
 - ❓ La columna **Precio** es editable y actualiza en caliente
 - ❓ **Abonos:**
 - ❓ Tabla de precios por combinación **Categoría + Duración**.
 - ❓ Mismo patrón: claves de tarifa en gris (no editable) y precio editable.

En resumen: es una pantalla de administración que permite mantener tarifas por recinto con claridad sobre qué campos son estructura (clave) y cuáles son modificables (precio).

5) QuickTix.Desktop.SalesView

ID	Fecha de venta	Día Semana	Recinto	Vendido por	Cliente	Categoría	Pri
1023	08/01/2026 20:40	jueves	Piscina Municipal de Nalda	Gestor Piscina	epas	Niño	232.00
1022	08/01/2026 20:14	jueves	Piscina Municipal de Nalda	Gestor Piscina	epas	Niño	232.00
21	07/01/2026 18:35	miércoles	Piscina Municipal de Nalda	Gestor Piscina	epa	Niño	45.00
20	07/01/2026 18:34	miércoles	Piscina Municipal de Nalda	Gestor Piscina	epa	Jubilado	12.00
19	07/01/2026 18:30	miércoles	Piscina Municipal de Nalda	Gestor Piscina	epa	Adulto	25.00
13	02/01/2026 14:20	viernes	Piscina Municipal de Nalda	Gestor Piscina	josecito	Adulto	25.00
12	02/01/2026 14:12	viernes	Piscina Municipal de Nalda	Gestor Piscina	Cliente 2	Adulto	25.00
11	02/01/2026 14:07	viernes	Piscina Municipal de Nalda	Gestor Piscina	Cliente 2	Adulto	25.00
10	02/01/2026 14:01	viernes	Piscina Municipal de Nalda	Gestor Piscina	josecito	Adulto	25.00
9	27/12/2025 16:14	sábado	Piscina Municipal de Nalda	Gestor Piscina	sgsdgsrrrr	Adulto	25.00
8	27/12/2025 16:11	sábado	Piscina Municipal de Nalda	Gestor Piscina	Cliente 2	Adulto	25.00
7	27/12/2025 16:09	sábado	Piscina Municipal de Nalda	Gestor Piscina	Cliente 1	Niño	15.00
6	27/12/2025 16:07	sábado	Piscina Municipal de Nalda	Gestor Piscina	sgsdgsrrrr	Adulto	25.00
5	27/12/2025 15:55	sábado	Piscina Municipal de Nalda	Gestor Piscina	josecito	Niño	15.00
4	27/12/2025 15:54	sábado	Piscina Municipal de Nalda	Gestor Piscina	josecito	Jubilado	12.00
3	27/12/2025 15:52	sábado	Piscina Municipal de Nalda	Gestor Piscina	Cliente 2	Adulto	25.00
2	06/11/2025 20:38	jueves	Piscina Municipal de Nalda	Gestor Piscina	Cliente 1	Adulto	25.00

Propósito: consulta y auditoría de **histórico de ventas**, separando ventas de entradas y ventas de suscripciones.

Qué hace:

- ❓ Organiza el histórico en dos pestañas:
 - ❓ **Entradas**
 - ❓ **Suscripciones**
- ❓ En ambas, hay un botón **Recargar** que ejecuta el LoadCommand del submódulo correspondiente.
- ❓ **Tab Entradas:**
 - ❓ Listado principal con: Id, fecha/hora, día de semana, recinto, gestor, nº entradas, total.
 - ❓ Incluye un **Expander “Detalle de la venta”** que muestra:
 - ❓ cabecera de detalle (DetailHeader),

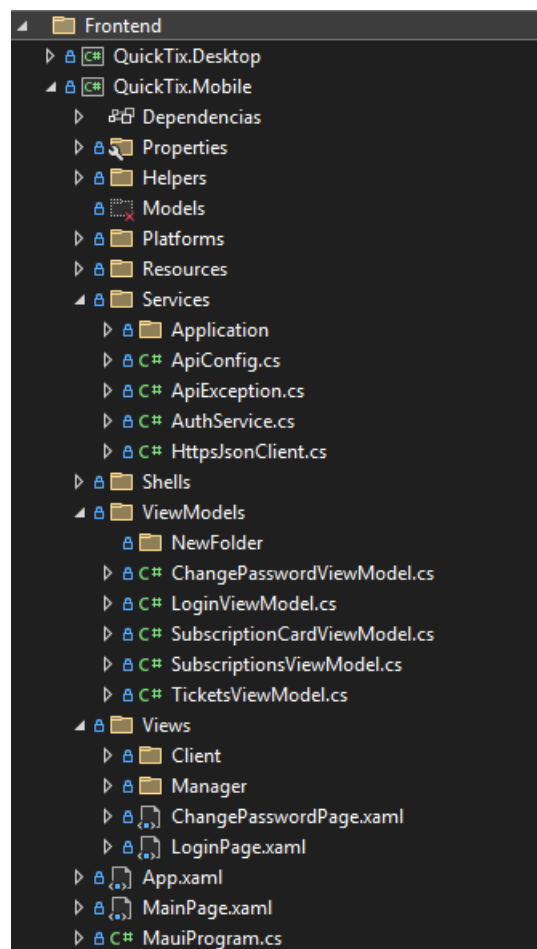
- ❓ líneas con tipo/contexto/cantidad/precio/total,
- ❓ campo “Invitado por” (se muestra a nivel de detalle, lo que sugiere que ciertas ventas de entradas pueden estar asociadas a un cliente invitador).
- ❓ Botón “Cerrar detalle” para ocultar el expander (control explícito de visibilidad).

❓ **Tab Suscripciones:**

- ❓ Listado de ventas de abonos con: Id, fecha/hora, día semana, recinto, gestor, cliente, categoría y precio.

Esta vista está orientada a control: revisar operaciones realizadas, identificar quién vendió, cuándo, en qué recinto, y con qué importe.

5.3.6 Frontend.Mobile



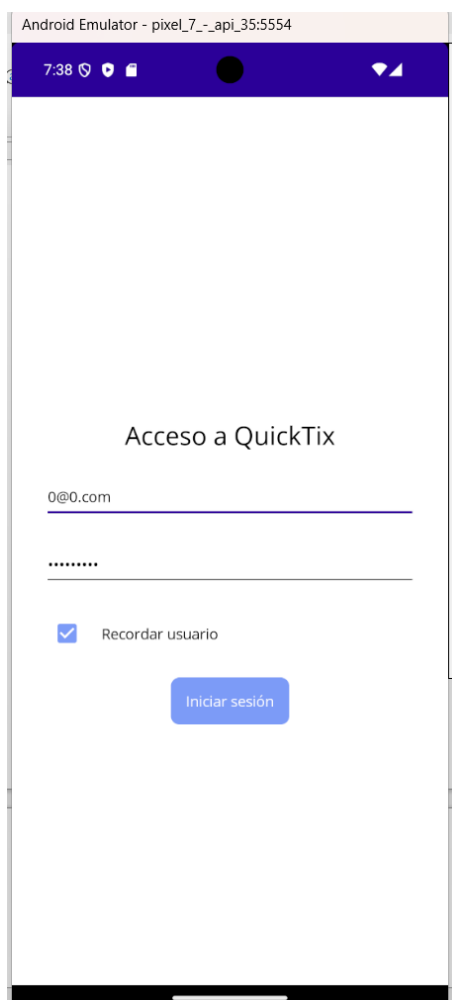
La capa **QuickTix.Mobile** es el **cliente móvil** de QuickTix, diseñado para cubrir escenarios de uso en movilidad (operativa en recinto) y acceso rápido desde smartphone. Su responsabilidad principal es ofrecer una experiencia “de operación” simplificada, consumiendo la API mediante servicios HTTP comunes, manteniendo estado de sesión (token/usuario) y aplicando validaciones de formulario y feedback de estado (cargando, errores, mensajes).

A nivel de navegación, la aplicación se organiza mediante **Shells separados por rol**, lo que permite:

- ❑ presentar menús y rutas diferentes según el perfil autenticado,
- ❑ restringir accesos por diseño (además de la autorización real en API),
- ❑ y reducir complejidad al no mezclar pantallas que no aplican a un rol concreto.

En el árbol del proyecto se aprecia esta separación en Shells y en Views/Client y Views/Manager, lo que sugiere una navegación distinta para clientes finales frente a gestores/operadores.

1) Views/LoginPage.xaml



Propósito: acceso inicial a la aplicación y creación de sesión.

Qué hace:

- ❑ Permite introducir **Email/usuario** y **contraseña**.
- ❑ Incluye opción “**Recordar usuario**”, para reducir fricción en accesos recurrentes.
- ❑ Lanza el comando CheckLoginCommand, que típicamente:
 - ❑ valida credenciales contra la API,
 - ❑ guarda token/sesión local,
 - ❑ y redirige al Shell correspondiente según rol.

2) Views/ChangePasswordPage.xaml

Propósito: forzar el cambio de contraseña en el primer inicio de sesión o ante políticas de seguridad.

Qué hace:

- ❑ Recoge **contraseña actual, nueva contraseña y confirmación**.
- ❑ Habilita el botón “Aceptar” solo si el formulario es válido (IsSubmitEnabled).
- ❑ Ejecuta ConfirmChangePasswordCommand, que normalmente:
 - ❑ llama a la API para actualizar credenciales,
 - ❑ actualiza el estado local de sesión si procede,
 - ❑ y permite continuar el flujo normal (Shell del rol).

3) Views/TicketsPage.xaml

Android Emulator - pixel_7_-_api_35:5554

7:44

Tickets Cerrar sesión

Datos de sesión
VenuelId=1 | ManagerId=3007 | Role=manager

Nueva línea

Tipo de ticket Contexto

Cantidad 1 Precio Opcional (Ej: 12.5)

Añadir línea

Líneas de la venta Líneas: 0 | Entradas: 0 | Total (si hay precios): 0

Aún no hay líneas añadidas.
Añade una línea para verla aquí.

Vender (batch) Limpiar

Propósito: pantalla operativa para **registrar una venta de entradas (batch)** desde móvil.

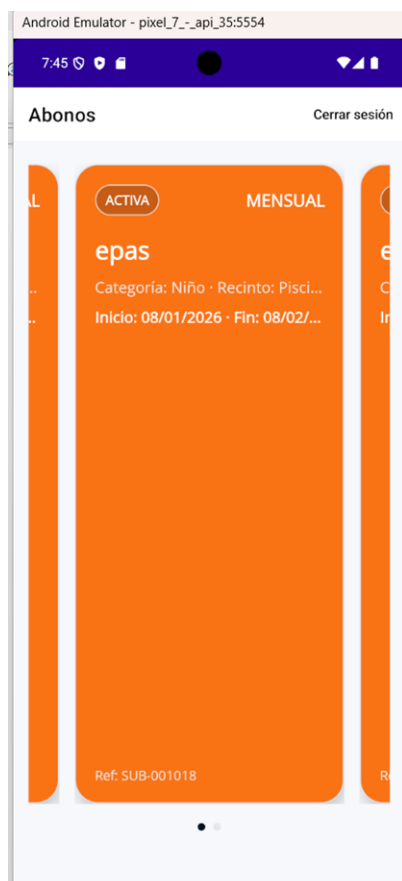
Qué hace:

- ❑ Muestra información de sesión (SessionInfo) para contexto del operador (quién está logueado, recinto, etc.).
- ❑ Permite construir una venta por **líneas**, con un flujo guiado:
 - ❑ selección de **Tipo de ticket y Contexto** (pickers),
 - ❑ entrada de **Cantidad**,
 - ❑ entrada opcional de **Precio unitario** (permite override/control manual si el caso lo requiere),
 - ❑ acción “**Añadir línea**” (solo habilitada si el formulario está completo, CanAddLine).
- ❑ Lista de líneas añadidas (Lines) con:
- ❑ visualización resumida (título, info y total),

- ❑ opción de **eliminar** mediante swipe o botón “Quitar”.
- ❑ Bloque inferior condicionado: **Invitado abonado** (RequiresClientId), donde se solicita ClientId cuando el contexto/tipo requiere asociar la venta a un abonado que invita.
- ❑ Barra inferior con acciones:
 - ❑ **“Vender (batch)”** (SellBatchCommand) para enviar la venta completa a la API,
 - ❑ **“Limpiar”** (ResetCommand) para reiniciar el formulario.
- ❑ Incluye **indicadores de estado** (IsBusy, StatusMessage) y **cerrar sesión** desde toolbar.

Esta vista es, en la práctica, el “punto de venta” móvil: rápida, orientada a velocidad, y preparada para minimizar errores en horas punta.

4) Views/SubscriptionsPage.xaml



Propósito: consulta visual de **abonos** en formato “tarjeta” y navegación rápida entre ellos.

Qué hace:

- ❑ Presenta las suscripciones en un **CarouselView**, con tarjetas ricas (estilo credencial):
 - ❑ estado (StatusText),
 - ❑ título (categoría/duración),
 - ❑ cliente asociado (ClientName),
 - ❑ validez (rango de fechas) y texto de referencia/identificador.
- ❑ Incluye tratamiento visual de **caducadas** (IsExpired) con color específico y etiqueta “CADUCADA”.
- ❑ Muestra errores generales (ErrorMessage) y permite **cerrar sesión** desde toolbar.

Esta vista está orientada a consulta/validación rápida: navegar entre abonos y comprobar estado y validez sin entrar en pantallas complejas.

6. Bases de datos y modelo de dominio

El dominio se articula en torno a recintos (venues), catálogo de tickets/abonos, pricing por recinto y registro de ventas.

6.1 Entidades principales

Entidad	Propósito	Relaciones	Notas / estado
Venue	Recinto/piscina. Punto de venta y catálogo.	1-N con tickets y abonos; 1-N con ventas.	Incluye IsActive como flag de publicación.
Ticket	Tipo de entrada (adulto/niño, etc.).	Se vende en contexto; precio depende de venue.	Ticket.Context puede afectar pricing (ej.: invitado abonado).
Subscription	Abono (por categoría y duración).	Asociado a ventas; puede tener precio fijo o por venue.	Precio provisional en DTO; pendiente unificación con pricing por venue.
Sale	Cabecera de venta.	N-1 Venue; N-1 Manager; 1-N SaleItems.	Incluye histórico y operaciones batch.
SaleItem	Detalle de venta.	N-1 Sale; referencia a Ticket/Subscription.	Pendiente de clarificar reglas de integridad y restricciones.
Pricing (VenuePriceMap)	Mapa de precios por venue.	1-1 por Venue con colecciones de precios.	Módulo en evolución: migraciones y endpoints específicos.

6.2 Integridad y reglas de negocio

- ❑ Validación de venta: consistencia entre items, venue y pricing vigente.
- ❑ Restricciones por roles: operaciones administrativas vs operativas.
- ❑ Contexto de ticket: descuentos o reglas especiales (ej. invitado abonado).
- ❑

6.3 Migraciones y semillas

El sistema está preparado para migraciones EF Core y semilla inicial de datos (p. ej., usuario admin y venues base). Pendiente: inventario formal de migraciones y un diagrama ER actualizado.

7. API REST

7.1 Rutas Endpoint (ApiRoutes)

Las rutas se centralizan para evitar duplicidad y facilitar mantenimiento. Listado derivado de la clase de constantes ApiRoutes.

Recurso	Operación	Ruta
Admin	GetAll	/api/Admin
Admin	Create	/api/Admin
Admin	GetById	/api/Admin/{id:int}
Admin	Update	/api/Admin/{id:int}
Admin	Delete	/api/Admin/{id:int}
Client	GetAll	/api/Client
Client	Create	/api/Client
Client	GetById	/api/Client/{id:int}
Client	Update	/api/Client/{id:int}
Client	Delete	/api/Client/{id:int}
Manager	GetAll	/api/Manager
Manager	Create	/api/Manager
Manager	GetById	/api/Manager/{id:int}
Manager	Update	/api/Manager/{id:int}
Manager	Delete	/api/Manager/{id:int}
Pricing	GetVenuePriceMap	/api/Pricing/venue/{venueid:int}
Pricing	UpsertVenuePriceMap	/api/Pricing/venue/{venueid:int}
Sale	GetAll	/api/Sale
Sale	Create	/api/Sale
Sale	HistorySubscriptions	/api/Sale/history/subscriptions
Sale	HistoryTickets	/api/Sale/history/tickets

Sale	HistoryTicketDetail	/api/Sale/history/tickets/{saleId:int}/detail
Sale	SellSubscription	/api/Sale/sell/subscription
Sale	SellTickets	/api/Sale/sell/tickets
Sale	SellTicketsBatch	/api/Sale/sell/tickets/batch
Sale	GetById	/api/Sale/{id:int}
Sale	Update	/api/Sale/{id:int}
Sale	Delete	/api/Sale/{id:int}
SaleItem	GetAll	/api/SaleItem
SaleItem	BySale	/api/SaleItem/by-sale/{saleId:int}
SaleItem	Subscriptions	/api/SaleItem/subscriptions
SaleItem	Tickets	/api/SaleItem/tickets
SaleItem	GetById	/api/SaleItem/{id:int}
Subscription	GetAll	/api/Subscription
Subscription	Create	/api/Subscription
Subscription	ByClient	/api/Subscription/by-client/{clientId:int}
Subscription	GetById	/api/Subscription/{id:int}
Subscription	Update	/api/Subscription/{id:int}
Subscription	Delete	/api/Subscription/{id:int}
Ticket	GetAll	/api/Ticket
Ticket	Create	/api/Ticket
Ticket	GetById	/api/Ticket/{id:int}
Ticket	Update	/api/Ticket/{id:int}
Ticket	Delete	/api/Ticket/{id:int}
User	GetAll	/api/User
User	ChangePassword	/api/User/change-password
User	Login	/api/User/login

User	Register	/api/User/register
User	GetById	/api/User/{id}
Venue	GetAll	/api/Venue
Venue	Create	/api/Venue
Venue	GetById	/api/Venue/{id:int}
Venue	Update	/api/Venue/{id:int}
Venue	Delete	/api/Venue/{id:int}

8. Implementación, configuración y despliegue

8.1 Compilación y ejecución local

En entorno local, QuickTix se ejecuta de forma distribuida: una **API ASP.NET Core** que conecta a **SQL Server**, y dos clientes (**WPF Desktop** y **.NET MAUI Mobile**) que consumen la API mediante una URL base configurable. El objetivo del procedimiento es disponer de una base de datos reproducible, aplicar migraciones y arrancar el sistema con datos mínimos (seed) para poder autenticarse y validar flujos.

8.1.1 Despliegue de SQL Server en Docker Desktop

Para simplificar la puesta en marcha y evitar instalaciones locales de SQL Server, se utiliza un contenedor oficial de Microsoft en Docker. El siguiente comando crea y arranca un SQL Server Developer Edition con persistencia en volumen:

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=Abcd123!" -e
"MSSQL_PID=Developer" -p 5000:1433 --name SQL_Server_QuickTix -v
SQL_Server_Volume:/var/opt/mssql -d mcr.microsoft.com/mssql/server:2022-latest
```

8.1.2 Aplicación de migraciones EF Core

Una vez SQL Server está operativo, se deben aplicar las migraciones para crear/actualizar el esquema de base de datos. Para ello se usa EF Core CLI: dotnet ef.

Instalación de la herramienta (si no está disponible)

1. Comprobar si está instalada:

```
dotnet ef --version
```

2. Si el comando no existe, instalar la herramienta global:

```
dotnet tool install --global dotnet-ef
```

3. Si ya estaba instalada pero quieres actualizarla:

```
dotnet tool update --global dotnet-ef
```

Nota práctica: tras instalar herramientas globales puede ser necesario reiniciar la terminal para que el PATH se actualice correctamente.

Ejecutar migraciones

Desde la raíz de la solución (o ajustando rutas según tu estructura), el patrón habitual es:

- ❑ El **proyecto que contiene las migraciones/DbContext** suele ser QuickTix.DAL.
- ❑ El **proyecto de arranque** suele ser QuickTix.API (donde están appsettings/DI).

Ejemplo genérico:

```
dotnet ef database update --project QuickTix.DAL --startup-project QuickTix.API
```

Con esto:

- ❑ EF Core carga la configuración del proyecto de arranque (API),
- ❑ encuentra el DbContext y las migraciones en DAL,
- ❑ y aplica las migraciones pendientes contra la base de datos indicada por la connection string.

Connection string esperada (orientativa)

Dado el mapeo de puertos -p 5000:1433, el servidor desde el host es:

- ❑ Server=localhost,5000;User Id=sa;Password=Abcd123!;TrustServerCertificate=True;

La base de datos puede depender del nombre configurado en appsettings.json. En muchos casos se usa Database=QuickTix o similar.

8.1.3 Arranque de la API y seed inicial

Con la base de datos preparada, se ejecuta la API en perfil Development. En el primer arranque, la aplicación ejecuta un **seed de datos mínimos** (por ejemplo usuarios/roles iniciales) para que el sistema sea utilizable desde el inicio sin configuraciones manuales.

Esto permite iniciar sesión directamente con las credenciales del usuario administrador predefinido (según el seed configurado), acceder al cliente Desktop/Mobile y comenzar a validar flujos.

Resultado esperado tras el primer arranque:

- ❑ Base de datos creada y actualizada con el esquema más reciente (migraciones aplicadas).
- ❑ Roles y usuario administrador iniciales insertados por el seeder.

- ❑ Autenticación operativa para entrar desde Desktop/Mobile con las credenciales de admin.

8.1.4 Ejecución de clientes

- ❑ **Desktop (QuickTix.Desktop)**: se ejecuta apuntando a la URL base de la API (por configuración). Una vez autenticado, habilita pantallas de gestión (usuarios, clientes, pricing, históricos).
- ❑ **Mobile (QuickTix.Mobile)**: se ejecuta en emulador/dispositivo. Requiere que la URL base de la API sea accesible desde ese entorno:
 - ❑ emulador Android no siempre resuelve localhost como el host del PC; normalmente se usa la IP local de la máquina o un alias específico del emulador.

9. Conclusiones y trabajo futuro

QuickTix alcanza el objetivo principal que se persigue en un proyecto académico con orientación práctica: construir una solución realista, estructurada y ampliable, con una base técnica coherente y un flujo operativo funcional. El resultado es una plataforma que ya dispone de los componentes esenciales de una aplicación “corriente” desplegable en un entorno real: **backend centralizado, persistencia relacional, contratos compartidos y clientes con patrón MVVM**, lo que permite que el sistema sea mantenible y evolucione sin necesidad de reescrituras completas.

A lo largo del desarrollo se han identificado más necesidades y posibles mejoras de las que ha sido viable implementar dentro del alcance y el tiempo disponibles. En un proyecto con origen en un problema real (venta y control en un recinto público), es habitual que aparezcan nuevas casuísticas y se detecten oportunidades de mejora continuamente. Aun así, el balance final es positivo: se ha conseguido una primera versión operativa que cubre flujos críticos (configuración y venta/consulta), y que además está soportada por decisiones arquitectónicas que facilitan el crecimiento del sistema.

De forma particular, el proyecto deja una base sólida en tres aspectos:

- ❑ **Arquitectura y mantenibilidad**: la separación por capas (API, Core, DAL, Contracts y clientes) reduce acoplamientos y clarifica responsabilidades. Esto permite incorporar nuevas funcionalidades sin “contaminar” el núcleo con detalles de UI o infraestructura.
- ❑ **Consistencia de comunicación**: el uso de contratos compartidos y respuestas estandarizadas facilita que Desktop y Mobile consuman la API de forma estable, y simplifica la gestión de errores y diagnósticos.
- ❑ **Operatividad del flujo de trabajo**: el sistema puede ejecutarse en local con despliegue reproducible de base de datos (contenedor SQL Server) y aplicación de migraciones, permitiendo una puesta en marcha rápida y coherente. Esta característica es especialmente importante si el objetivo es avanzar hacia un despliegue real.

9.1 Trabajo futuro y líneas de mejora

El siguiente salto de calidad para QuickTix no pasa únicamente por añadir más código, sino por consolidar y pulir el producto como solución completa. Las mejoras previstas se pueden agrupar en tres bloques.

1) Consolidación técnica (calidad y robustez)

- ❑ **Refuerzo de servicios de negocio:** consolidar lógica en Core/Servicios para evitar que reglas sensibles (como pricing, validaciones o restricciones por rol/recinto) queden dispersas o duplicadas entre clientes y API.
- ❑ **Documentación de despliegue y reporte completa:** convertir el arranque local en un procedimiento totalmente guiado (scripts, variables, capturas y checklist), y preparar un flujo de despliegue a producción más directo (por ejemplo, con Docker Compose, configuración por entorno y documentación de networking/URLs).
- ❑ **Pruebas automatizadas:** aumentar cobertura de unit tests y tests de integración para endpoints críticos (login, venta batch, pricing, cancelaciones, históricos). Esto permitiría iterar con mayor seguridad cuando se introduzcan nuevas reglas o cambios de modelo.

2) Mejora estética y experiencia de usuario (prioridad en Mobile)

Aunque la parte de administración de escritorio está más completa y orientada a productividad (listados, formularios, paneles), la aplicación móvil debe evolucionar hacia una experiencia más pulida y orientada al usuario final. Algunas líneas claras son:

- ❑ **Mejorar consistencia visual:** jerarquía tipográfica, espacios, componentes reutilizables, estados vacíos, transiciones y feedback.
- ❑ **Refinar los flujos de operación:** reducir aún más fricción en la venta, confirmaciones claras, manejo de errores más guiado y mejoras en accesibilidad.
- ❑ **Diseño orientado a escenarios reales:** optimizar pantallas para uso con una sola mano, rapidez en horas punta y claridad de información.

3) Nuevas funcionalidades centradas en el cliente

El mayor margen de crecimiento funcional está en el lado “cliente”, especialmente en la parte móvil:

- ❑ **Área de cliente más completa:** consulta de abonos con detalle, histórico de compras, estado de validez, y facilidades para identificación/validación en acceso.
- ❑ **Gestión avanzada de abonos:** renovaciones, avisos de caducidad, reglas de uso y, si aplica, invitaciones asociadas a abonados.

- ❓ **Mejoras operativas:** funcionalidades que aporten valor inmediato al recinto, como filtros avanzados de ventas, exportación, cierres diarios, o estadísticas básicas.

En conclusión, aunque el alcance podría haberse ampliado significativamente (especialmente en estética y funcionalidades de cliente), QuickTix logra un hito importante: una versión inicial que ya es operativa y con fundamentos técnicos suficientemente sólidos como para evolucionar hacia una aplicación desplegable en producción. El trabajo futuro se centra en convertir esa base en un producto más completo: más robusto, mejor documentado y con una experiencia de usuario más cuidada, especialmente en el cliente móvil.